Q1 – Building Text Under Load: String, StringBuilder or StringBuffer? (6 Marks, K4)

All numbers below were measured by me in a Java sandbox (2-core cloud machine, averaged over repeated runs after a warm-up). Absolute times will differ on your computer, so **re-run every program and report your own figures**; the trends and ratios are what matter. Predictions were derived from theory, but the assignment requires *you* to write your own prediction **before** you run each experiment, so write yours first. Each screenshot must be linked to the test it shows.

1. Prediction (before implementation)

Approach	Predicted behaviour as fields and tickets increase	Reason
`String` + in a loop	Time grows **quadratically** with the number of fields n; many short-lived objects	`String` is immutable. Every `s += x` creates a *new* String and copies all characters built so far, so the total copy is L + 2L + … + nL = $O(n^2 L)$ characters for fields of length L, and one garbage String per iteration.
`StringBuilder`	Grows **linearly**, O(n L); few objects	It appends into one mutable char/byte array that doubles when full (amortised O(1) per character), so each character is copied about twice in total.
`StringBuffer`	Linear like `StringBuilder`, slightly slower	Same algorithm but every method is `synchronized`; the lock costs time even with one thread.
For one short ticket (10 fields)	All three are almost the same; the difference is negligible	n is so small that n^2 is tiny; the cost is dominated by fixed overhead.

2. Workloads

ID	Workload	Purpose
W1	1 ticket × 10 fields	the "short ticket" case
W2	10,000 tickets × 50 fields	the typical busy-server case
W3 (my own)	200 tickets × 500 fields	long tickets (e.g. group booking with many passengers)
Scaling	1 ticket with n = 100, 1,000, 5,000, 10,000, 20,000 fields	to see the growth curve

Fields are strings like `Field12=Value12;` (about 20 characters each).

3. Implementation (all programs ran)

```
import java.util.*;

public class TicketBench {
    static String[] fields(int n) { String[] f = new String[n]; for
(int i = 0; i < n; i++) f[i] = "Field" + i + "=Value" + i + ";";
return f; }

    static String viaString(String[] f) { String s = ""; for (String x
: f) s += x; return s; }
    static String viaBuilder(String[] f) { StringBuilder sb = new
StringBuilder(); for (String x : f) sb.append(x); return
sb.toString(); }
    static String viaBuffer(String[] f) { StringBuffer sb = new
StringBuffer(); for (String x : f) sb.append(x); return sb.toString();
}
    static String viaJoin(String[] f) { return String.join("", f); }
    static String viaJoiner(String[] f) { StringJoiner j = new
StringJoiner(""); for (String x : f) j.add(x); return j.toString(); }
    static String viaFormat(String[] f) { StringBuilder sb = new
StringBuilder(); for (String x : f) sb.append(String.format("%s", x));
return sb.toString(); }

    interface Builder { String build(String[] f); }
    static double time(Builder b, String[] f, int tickets, int runs) {
        long best = Long.MAX_VALUE; long sum = 0;
        for (int r = 0; r < runs; r++) {
            long t0 = System.nanoTime();
            int len = 0;
            for (int t = 0; t < tickets; t++) len +=
b.build(f).length();
            long dt = System.nanoTime() - t0; sum += dt;
            if (len == 0) System.out.println();
```

```
        }
        return sum / 1e6 / runs;
    }
    public static void main(String[] a) {
        // warm-up
        for (int i = 0; i < 2000; i++) { viaString(fields(10));
viaBuilder(fields(10)); viaBuffer(fields(10)); }
        int[][] workloads = {{1, 10}, {10000, 50}, {200, 500}};     //
{tickets, fields}
        String[] names = {"W1: 1 ticket x 10 fields", "W2: 10,000
tickets x 50 fields", "W3 (own): 200 tickets x 500 fields"};
        System.out.println("workload | String(+) | StringBuilder |
StringBuffer | String.join | StringJoiner | String.format   (ms, avg
of 7 runs)");
        for (int w = 0; w < workloads.length; w++) {
            String[] f = fields(workloads[w][1]); int t = workloads[w]
[0];
            System.out.printf("%-36s | %9.3f | %9.3f | %9.3f | %9.3f |
%9.3f | %9.3f%n", names[w],
                    time(TicketBench::viaString, f, t, 7),
time(TicketBench::viaBuilder, f, t, 7), time(TicketBench::viaBuffer,
f, t, 7),
                    time(TicketBench::viaJoin, f, t, 7),
time(TicketBench::viaJoiner, f, t, 7), time(TicketBench::viaFormat, f,
t, 7));
        }
        System.out.println("\nscaling in number of fields n (one
ticket, avg of 30 runs, microseconds):  n | String(+) |
StringBuilder");
        for (int n : new int[]{100, 1000, 5000, 10000, 20000}) {
            String[] f = fields(n);
            System.out.printf("%6d | %12.1f | %10.1f%n", n,
time(TicketBench::viaString, f, 1, 30) * 1000,
time(TicketBench::viaBuilder, f, 1, 30) * 1000);
        }
    }
}
```

4. Measured results

Time (ms for the whole workload, average of 7 runs)

Workload	`String +`	`StringBuilder`	`StringBuffer`	`String.join`	`StringJoiner`	`String.format` per field
W1: 1 × 10 fields	0.002	0.003	0.002	0.004	0.021	0.475
W2:	45.7	7.7	11.5	5.1	7.2	33.3

Workload	`String +`	`StringBuilder`	`StringBuffer`	`String.join`	`StringJoiner`	`String.format` per field
10,000 × 50						
W3: 200 × 500	54.8	1.2	1.3	0.7	0.9	6.5

W1 is a few microseconds, which is at the limit of what `System.nanoTime()` can resolve, so differences there are noise: **for one short ticket the choice does not matter**.

Growth with the number of fields n (one ticket, microseconds)

n	`String +`	`StringBuilder`	ratio
100	8.1	1.0	8×
1,000	1,122	10.0	112×
5,000	33,705	65.3	516×
10,000	135,470	118.2	1,146×
20,000	576,622	441.8	1,305×

Doubling n from 10,000 to 20,000 multiplied the + time by 4.3 (quadratic would give 4) and the `StringBuilder` time by 3.7 (roughly linear, with some noise): the predicted growth rates are confirmed.

Objects and memory (n = 5,000 fields, final length 97,780 chars, `MemoryAndObjects.java`)

```
public class MemoryAndObjects {
    static long used() { System.gc(); System.gc(); Runtime r =
Runtime.getRuntime(); return r.totalMemory() - r.freeMemory(); }
    public static void main(String[] a) {
        int n = 5000; String[] f = new String[n]; for (int i = 0; i <
n; i++) f[i] = "Field" + i + "=Value" + i + ";";
        // estimate of intermediate String objects: + creates a new
String (and a hidden builder/array) per iteration
        long total = 0, finalLen = 0; for (String x : f) { total +=
x.length(); }
        // characters copied by repeated +:  sum_{i=1..n} (length
after i fields)
        long copied = 0, len = 0; for (String x : f) { len +=
x.length(); copied += len; }
        System.out.println("n=" + n + " fields, final length=" + total
+ " chars");
        System.out.println("intermediate Strings created by +    : " +
n + " (one per iteration, each longer than the last)");
        System.out.println("characters copied by +  (sum of lengths):
" + copied + "  = about " + (copied / total) + " x the final length");
```

```
        System.out.println("characters copied by StringBuilder
(approx, with doubling): <= " + (2 * total) + "  = about 2 x the final
length");
        long before = used();
        String s = ""; for (String x : f) s += x;
        long after = used();
        System.out.println("heap kept after + loop   : " + (after -
before) / 1024 + " KB (only the final String survives; the
intermediates are garbage)");
        long b2 = used();
        StringBuilder sb = new StringBuilder(); for (String x : f)
sb.append(x); String s2 = sb.toString();
        long a2 = used();
        System.out.println("heap kept after builder  : " + (a2 - b2) /
1024 + " KB (builder + final String)");
        // allocation measured via ThreadMXBean
        com.sun.management.ThreadMXBean tm =
(com.sun.management.ThreadMXBean)
java.lang.management.ManagementFactory.getThreadMXBean();
        long id = Thread.currentThread().getId();
        long p0 = tm.getThreadAllocatedBytes(id);
        String t1 = ""; for (String x : f) t1 += x;
        long p1 = tm.getThreadAllocatedBytes(id);
        StringBuilder sb3 = new StringBuilder(); for (String x : f)
sb3.append(x); String t2 = sb3.toString();
        long p2 = tm.getThreadAllocatedBytes(id);
        System.out.println("bytes ALLOCATED by + loop      : " + (p1 -
p0) / 1024 + " KB");
        System.out.println("bytes ALLOCATED by StringBuilder: " + (p2
- p1) / 1024 + " KB");
    }
}
```

Measure	`String +`	`StringBuilder`
Intermediate String objects created (estimate)	**5,000** (one per iteration, each longer than the last)	1 final String (+ about 13 internal array resizings of the builder)
Characters copied	239,958,990 ≈ **2,454 × the final length** (theory: about n/2 = 2,500)	≤ 195,560 ≈ **2 × the final length**
Bytes allocated (`ThreadMXBean.getThreadAllocatedBytes`)	**234,541 KB** (≈ 229 MB)	**383 KB**
Heap still in use after the loop	33 KB (only the final String)	239 KB (builder + String)

So + leaves almost nothing behind but *allocates 600 times more memory* that the garbage collector must reclaim; the "memory after" number alone would hide this problem, which is why I also measured allocated bytes.

5. Prediction vs. observation

- Quadratic vs linear: **confirmed** (table above; 1,305× at n = 20,000).
- `StringBuffer` slightly slower than `StringBuilder`: **confirmed** (11.5 vs 7.7 ms in W2, 1.3 vs 1.2 ms in W3). With a single thread the cost is only the uncontended lock.
- Negligible difference for one short ticket: **confirmed** (W1).
- One surprise: in W2 (50 fields, 10,000 tickets) + is "only" 6× slower, not hundreds of times, because n = 50 is small and the JIT optimises small concatenations; the quadratic effect needs larger n (W3 and scaling table).

6. Design decision – multithreaded booking server

- **Per-ticket text: `StringBuilder`** (or `String.join` when all the pieces are already in an array). The builder is a *local variable* inside the method that creates the ticket, so only one thread ever touches it. It needs no lock, and the cheapest correct choice is the one without synchronisation.
- **Shared booking log: `StringBuffer`** (or, better, a thread-safe logging mechanism). Several threads append to the same object, and `StringBuilder` is not thread-safe. I demonstrated this with 4 threads × 50,000 appends of one character (`ThreadSafetyDemo.java`): expected length 200,000; `StringBuffer` gave **200,000** every time, and `StringBuilder` gave **199,590, 199,774 and 199,969** (lost updates; sometimes it can also throw `ArrayIndexOutOfBoundsException`).

```
public class ThreadSafetyDemo {
    public static void main(String[] a) throws Exception {
        int threads = 4, per = 50_000;
        StringBuilder sb = new StringBuilder(); StringBuffer sf = new
StringBuffer();
        Thread[] ts = new Thread[threads];
        int[] failed = {0};
        for (int i = 0; i < threads; i++) { ts[i] = new Thread(() -> {
for (int j = 0; j < per; j++) { try { sb.append("x"); } catch
(RuntimeException e) { synchronized (failed) { failed[0]++; } }
sf.append("x"); } }); ts[i].start(); }
        for (Thread t : ts) t.join();
        System.out.println("expected length " + (threads * per));
        System.out.println("StringBuffer  length = " + sf.length() + "
(always correct)");
        System.out.println("StringBuilder length = " + sb.length() + "
exceptions=" + failed[0] + "  (lost updates / corrupted when
shared)");
    }
}
```

- **Caveat:** `StringBuffer` makes each single `append` atomic, but a log *line* built from several appends (`append(time).append(pnr)…`) can still interleave with another

thread's line. For real logs, append a complete line with one call, or protect the multi-step sequence with `synchronized`, or use a logging library or a `ConcurrentLinkedQueue`.

7. Alternative concept and complexity analysis

`String.join()`, `StringJoiner` or `String.format()` instead of an explicit `StringBuilder`:

```
String ticket  = String.join(";", fields);                    // when the pieces are in an array/list
StringJoiner j = new StringJoiner(";", "[", "]"); for (String f : fields) j.add(f);
String line    = String.format("%s|%s|%s", pnr, name, coach);   // a fixed template with a few fields
```

`String.join` and `StringJoiner` use a builder internally, so they are also O(n L), and `String.join` knows the total length in advance so it allocates exactly once. It was the fastest in my tests (5.1 ms in W2, 0.7 ms in W3). `String.format` parses the format string on every call and creates a `Formatter`, so it is about 4–5× slower than a builder (33.3 vs 7.7 ms in W2): use it for readable templates, not inside hot loops.

Time to build a ticket of n fields of length L:

- Repeated +: iteration i copies the i·L characters built so far, so the total is $L(1 + 2 + \ldots + n) = L{\cdot}n(n+1)/2 = \mathbf{O(n^2\ L)}$ characters copied (measured: 239,958,990 copies for n = 5,000 against the predicted $L{\cdot}n^2/2 \approx 245$ million).
- `StringBuilder`: each character is written once, plus the copies when the capacity doubles (a geometric series totalling less than the final length again), so the total is about $2{\cdot}nL = \mathbf{O(n\ L)}$ characters.

Extra space created by intermediate Strings: with + the *peak live* memory is small (the previous and the new String, O(nL)), but the *total allocation* is $\mathbf{O(n^2\ L)}$ characters, so the garbage collector must reclaim $O(n^2\ L)$ bytes (measured 229 MB vs 383 KB). With `StringBuilder` the total allocation is O(nL) (the final array plus the discarded smaller arrays, less than twice the final size). The extra space at any moment is O(nL) in both, but the allocation *rate* differs by a factor of about n.